Project 2

Sigurd Riseth
Department of Physics
University of Oslo
Oslo, Norway
sigurri@uio.no

***Abstract*—abstract.**

***Index Terms*—classification, regression, neural networks, runge's function, MNIST**

I. Introduction

Warren McCulloch and Walter Pitts (1943) should probably be mentioned. Breakthroughs like Rosenblatt (1958), Minsky & Papert (1969) with the XOR problem and the first *AI-winter*, Rumelhart, Hinton & Williams (1986) with backpropagation. This all made way for later deep learning with CNN, GANs, Transformers, and much more.

The remainder of this report is structured as follows. Section II presents the ... Section III describes the data, the preprocessing, the code and how it was verified. Section IV presents and discusses the results in the order ..., and section V summarizes our findings and outlines future work. Appendix A declares our use of large language models. The code and results behind every figure and table, are available in our publicly hosted repository [1].

II. Method

This section presents the models, linear regression, logistic regression, and neural networks, and finally the evaluation methods used. We assume the reader has familiarity with basic linear algebra and statistics.

A. Linear regression

1) Mean squared error: The mean squared error (MSE)

2) Gradient descent:

B. Logistic regression

Logistic regression is a natural choice as a baseline. It is a simple model in which everything can be computed, its cost function is convex, and it is exactly a neural network with no hidden layers [2].

1) Binary cross entropy:

2) Categorical cross entropy:

C. Neural networks

Linear and logistic regression share a common limitation: both act on a fixed design matrix $\boldsymbol{X}$, so the model can only be as expressive as the features we choose to put into it. A feed-forward neural network removes this limitation by inserting one or more *hidden layers* between the input and the output, each applying a linear map followed by a non-linear activation. The final layer is then nothing more than a linear or logistic regression, but it acts on features that are *learned* from the data rather than given in advance [2].

1) The artificial neuron: An artificial neuron computes a linear predictor, as in logistic regression, followed by an activation function. We number layers as $l = 0, 1, \ldots, L$, with $l = 0$ the input and $l = L$ the output, and M_l the number of nodes in layer L. For a single neuron j in layer l, the activation is

$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \tag{1}$$

where w_{ij}^l is the weight connecting node i of layer $l-1$ to node j of layer l, and b_j^l is the bias of node j in layer l. We can then obtain the output by applying the activation function [2],

$$a_j^l = f(z_j^l). \tag{2}$$

This whole process is depicted in fig. 1, where the activation and output of a single neuron j in layer l is shown.

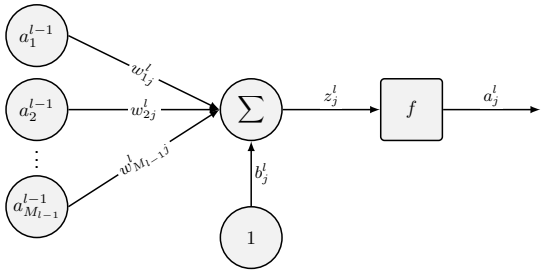


Fig. 1. The activation and output of a single neuron j in layer l, as shown in eqs. (1) and (2).

2) Forward pass: We can write the activation and output of all neurons the layer l in matrix-form, where eqs. (1) and (2) become

$$\boldsymbol{z}^l = (\boldsymbol{W}^l)^\top \boldsymbol{a}^{l-1} + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = \boldsymbol{f}(\boldsymbol{z}^l), \tag{3}$$

with $\boldsymbol{f}$ applied element-wise. Evaluating eq. (3) sequentially for all l-values is the feed-forward pass and returns the network output $\tilde{\boldsymbol{y}} = \boldsymbol{a}^L$ [2].

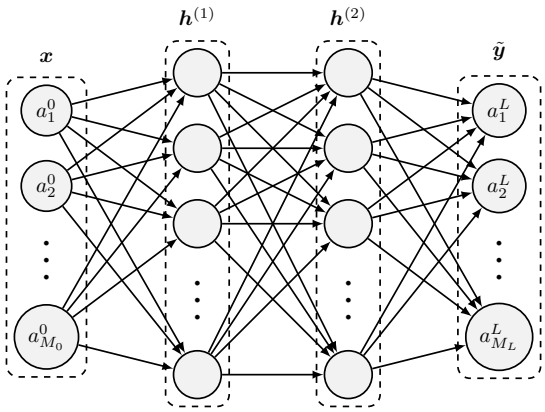


Fig. 2. Illustration of a fully connected feed-forward neural network. Adapted from Love [3].

We can further collect the weights and biases into a single parameter vector $\boldsymbol{\Theta} = \{\boldsymbol{w}^l, \boldsymbol{b}^l\}_{l=1}^L$, and write the optimization problem as

$$\hat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta}} C(\boldsymbol{\Theta}), \quad C(\boldsymbol{\Theta}) = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \tilde{y}_i) + \lambda\Omega(\boldsymbol{\Theta}), \tag{4}$$

where L is the cost function and Ω is the regularization term [2]. This is a non-convex optimization problem, so there is no closed-form solution, and we must resort to the gradient-based methods reported in section II-A2 to find a local minimum.

3) Activation functions: The activation function must be non-linear and differentiable almost everywhere. If it were linear, the composition of multiple layers would collapse into a single affine transformation, and the network would be no more expressive than a single layer [2]. Our study will revolve around the following three activation functions:

a) Sigmoid: The logistic or sigmoid function is given by

$$\sigma(z) = \frac{1}{1+e^{-z}}, \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)),$$

and was the standard choice for decades. Its main drawback is that it saturates for large $|z|$, which leads to vanishing gradients and slow training [2].

b) ReLU: The rectified linear unit (ReLU),

$$\mathrm{ReLU}(z) = \max(0, z), \quad \mathrm{ReLU}'(z) = \begin{cases} 1, & \text{if } z > 0, \\ 0, & \text{if } z \le 0, \end{cases}$$

is faster to compute, does not saturate for $z > 0$, and is a sound default choice for hidden layers. Its main drawback is that it can lead to dead neurons, which never activate and thus never learn [2].

The leaky ReLU (LReLU) repairs this issue by allowing a small gradient for negative values, so the gradient never completely vanishes. It is defined as

$$\mathrm{LReLU}(z) = \begin{cases} z, & \text{if } z > 0, \\ \alpha z, & \text{if } z \le 0, \end{cases} \quad \alpha \approx 0.01,$$

and its derivative is

$$\mathrm{LReLU}'(z) = \begin{cases} 1, & \text{if } z > 0, \\ \alpha, & \text{if } z \le 0. \end{cases}$$

4) Backpropagation: We can compute the gradient of the cost function with respect to the parameters using the back-propagation algorithm [4]. The key idea is to compute the error at the output layer and then propagate it backward through the network.

The error is given by

$$\boldsymbol{\delta}^l = (\boldsymbol{W}^{l+1}\boldsymbol{\delta}^{l+1}) \odot \boldsymbol{f}'(\boldsymbol{z}^l),$$

where $\odot$ denotes the element-wise product, and $\boldsymbol{\delta}^l$ is the error at layer l. For $l = L$, we don't have a next layer, but for the two natural pairings of output layer and loss function (i.e., linear output with MSE loss and sigmoid output with binary cross-entropy loss), we have $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$ [2]. The gradients with respect to the weights and biases are then given by

$$\frac{\partial C}{\partial \boldsymbol{W}^l} = \boldsymbol{\delta}^l \boldsymbol{a}^{l-1}, \qquad \frac{\partial C}{\partial \boldsymbol{b}^l} = \boldsymbol{\delta}^l.$$

Finally, we can update the parameters using gradient descent:

$$\boldsymbol{W}^l \leftarrow \boldsymbol{W}^l - \gamma\frac{\partial C}{\partial \boldsymbol{W}^l}, \qquad \boldsymbol{b}^l \leftarrow \boldsymbol{b}^l - \gamma\frac{\partial C}{\partial \boldsymbol{b}^l},$$

where γ is the learning rate.

5) Choice of output layer and loss function:

6) Weight initialization:

```
if initialization == "he":
    std = np.sqrt(2.0 / n_inputs)
elif initialization in ("xavier", "glorot"):
    std = np.sqrt(2.0 / (n_inputs + n_neurons))
```

7) Regularization: A network with more parameters than data points available can fit the entire training set exactly, so applying regularization is essential [2].

D. Evaluation

III. Implementation

All methods were implemented from scratch in Python using NumPy [5]. The source code, unit tests, and experiment scripts are available in the project repository [1]. This section describes the datasets and preprocessing, the organization of the implementation, and the procedures used to verify correctness.

A. Data and preprocessing

For the regression task, we use Runge's function [6], a classical example for studying the bias–variance trade-off. It is defined as

$$f(x) = \frac{1}{1+25x^2}, \qquad x \in [-1, 1]. \tag{5}$$

As observed in Project 1 [7], polynomial regression can exhibit substantial oscillations near the endpoints of this interval. We generate $n = 100$ input samples uniformly from $[-1, 1]$, evaluate the function at these inputs, and add i.i.d. Gaussian

noise with standard deviation $\sigma = 0.1$ to obtain the observed targets.

For the classification task, we use the MNIST dataset [8], which contains 70,000 grayscale images of handwritten digits, each of size 28×28 pixels, with labels corresponding to the ten classes $0, \ldots, 9$. [Describe the train/test split and any preprocessing applied to the images, such as flattening, normalization, or label encoding.]

B. Code structure

We reuse the structure and the gradient machinery of the previous project [7]. The code is structured in a package `fys_stk4155_p2`, containing the directory `nn`; which holds the code for the models including activation functions, layers (`Dense`, `Linear`, etc.), the network class `FFNN`, and loss functions.

Thin constructor of the `FFNN` for logistic regression, which returns a single layer with softmax activation and cross entropy loss. This allows us to use this as a baseline model when evaluating the later neural networks.

C. Verification

IV. Results and Discussion

This section presents and discusses the results from our experiments. First we cover the regression task, comparing linear regression to regression using FFNNs; afterward we cover the classification task of classifying handwritten digits using neural networks.

A. Regression

1) Old-school approach: We compare the best OLS model from Project 1 with two FFNN architectures. The resulting predictions are shown in fig. 3. The OLS model exhibits pronounced oscillations near the boundaries of the input domain, whereas both neural networks provide smoother approximations of the underlying function. This illustrates one limitation of polynomial regression: its predictions can become unstable near the boundaries, particularly when using higher-degree polynomials. In contrast, the FFNNs learn the function directly from the input features, without requiring us to specify a polynomial design matrix. The input to the neural networks is therefore simply the one-dimensional feature matrix $\boldsymbol{X} \in \mathbb{R}^{n \times 1}$.

The quantitative results are presented in table I. The FFNN with two hidden layers of 100 neurons achieves the lowest MSE relative to the true function, outperforming the one-hidden-layer FFNN and OLS by factors of approximately 2 and 10, respectively.

Interestingly, OLS achieves the lowest training MSE, but this does not translate into the best approximation of the underlying function. The discrepancy between its training and test performance, together with the oscillations visible in fig. 3, suggests that the polynomial model is sensitive to the noise in the training data and exhibits overfitting. The FFNN models provide a better approximation of the true function, despite both having a slightly higher training MSE.

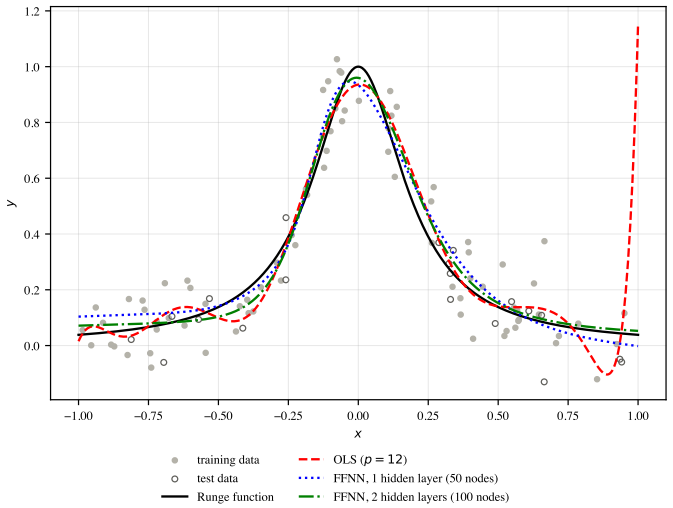


Fig. 3. Comparison of OLS and two FFNN architectures when approximating Runge's function. The data consist of 100 noisy samples with $\sigma = 0.1$, with 20% reserved for testing. Both FFNNs were trained for 1000 epochs using RMSProp, a learning rate of 0.001, and a batch size of 32.

TABLE I
Training and test performance of OLS and two FFNN architectures on Runge's function.

	train MSE	test MSE	train R^2	test R^2	MSE vs f
OLS ($p = 12$)	0.0081	0.0086	0.9207	0.6280	0.0125
FFNN (50)	0.0097	0.0092	0.9053	0.5996	0.0020
FFNN (100, 100)	0.0089	0.0085	0.9136	0.6315	0.0012

The cost of choosing a larger neural network is shown in fig. 4, where we can see how the larger models gets stuck at a plateau around 10^{-1} for a large period of the training. At this plateau, the model is predicting the mean of Runges function, and the gradient pointing towards useful curved features is tiny. This is the well-known plateau behavior of sigmoid networks [9], which can be avoided by using ReLU, or its variants, or applying batch normalization.

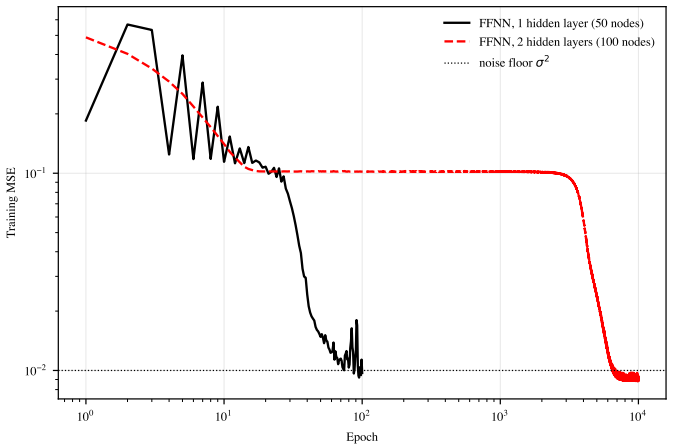


Fig. 4. Training history for both FFNN models.

Meanwhile, in fig. 5, we can see how Adam and RMSProp escape much sooner than plain gradient descent at the same learning rate. Additionally, a higher learning rate escapes the plateau faster and reaches the optimum faster but with noticeably more noise. This is a clear advocate for learning rate scheduling which we will use in later parts of this project.

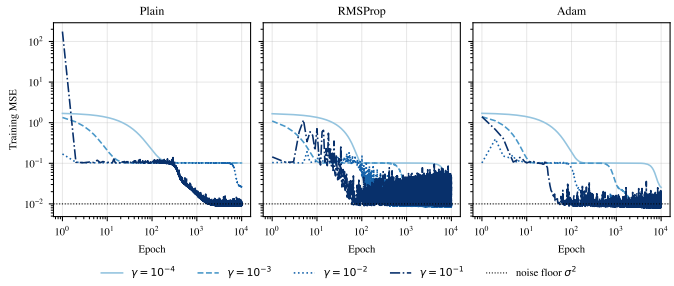


Fig. 5. Optimizer curves.

2) The ReLU-family, larger networks and overfitting: Now we explore how the ReLU and its Leaky ReLU variant perform, and how larger networks ...

V. Conclusion

TODO: conclusion.

References

[1] S. Riseth, *FYS-STK4155 Project 2*, version v0.1.0, GitHub, Nov. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P2

[2] M. Hjorth-Jensen, *Machine Learning and the Physical Sciences: From Discriminative Methods to Generative Methods and Reinforcement Learning*. Springer, Oct. 9, 2026. [Online]. Available: https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf

[3] F. Love, *NNTikZ: TikZ Diagrams for Deep Learning and Neural Networks*, 2024. Accessed: Oct. 15, 2026. [Online]. Available: https://github.com/fraserlove/nntikz

[4] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, **323**, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0 Accessed: Oct. 27, 2026. [Online]. Available: https://www.nature.com/articles/323533a0

[5] C. R. Harris et al., "Array programming with NumPy," *Nature*, **585**, no. 7825, pp. 357–362, Sep. 17, 2020, ISSN: 0028-0836, 1476-4687. DOI: 10.1038/s41586-020-2649-2 Accessed: Aug. 23, 2026. [Online]. Available: https://www.nature.com/articles/s41586-020-2649-2

[6] C. Runge, "Über empirische Funktionen und die Interpolation zwischen äquidistanten Ordinaten," *Zeitschrift für Mathematik und Physik*, **46**, pp. 224–243, 1901.

[7] S. Riseth, *FYS-STK4155 Project 1*, version v0.1.0, GitHub, Oct. 2026. [Online]. Available: https://github.com/sigurdriseth/FYS-STK4155-P1

[8] Y. LeCun, C. Cortes, and C. J. Burges, "MNIST handwritten digit database," *ATT Labs [Online]. Available: http://yann.lecun.com/exdb/mnist*, **2**, 2010.

[9] A. M. Saxe, J. L. McClelland, and S. Ganguli. "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks." arXiv: 1312.6120 [cs.NE], Accessed: Nov. 2, 2026. [Online]. Available: http://arxiv.org/abs/1312.6120 pre-published.

Appendix

A. Use of AI/LLM tools

Large language models were used throughout this project, following the course guidelines[1], which also define the contribution levels below. All code was tested, all derivations were checked, and every claim in the report was read, understood and verified by the author.

1) Tools used:

- TODO: tools, models, and dates.

TODO: summary of how each tool was used.

2) Text writing and editing: Table II gives the contribution level for each part of the report.

TABLE II
LLM CONTRIBUTION TO TEXT WRITING AND EDITING (LEVELS 0–3).

Section	Level	Notes
Abstract	TODO	
Introduction	TODO	
Methods	TODO	
Implementation	TODO	
Results and discussion	TODO	
Conclusion	TODO	
Appendices	TODO	

3) Code generation and assistance: Table III gives the level for each source file (levels 0–4). Functions with level 2 or above carry an `LLM-assisted` tag in their docstring.

4) Declared and undeclared use: TODO: one example of a use that must be declared, and one example of a use that need not be declared (e.g. using an LLM as a tutor and then writing about it in your own words).

B. Additional results

TODO: additional results.

[1] https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/LLM_Usage_Declaration_Guidelines.md

TABLE III
LLM CONTRIBUTION TO THE CODE (LEVELS 0–4).

File	Level	Description
`b-regression-ffnn.ipynb`	3	Refactoring code and markdown cells for readability and write the code for producing fig. 3. Parallelize the CV search for optimal hyperparameters.